PRD: Notification Event Tracking Table

Backend / Data Platform — Notifications

Field	Value
Owner	Mohammad Rushan
Date	July 08, 2026
Related to	Backend / Data platform — Notifications

1. Overview

We currently cannot answer basic questions about our notification system — which notifications get seen, which get clicked, which fail, and whether behavior differs by user segment (gender, premium vs free, active vs dormant, device brand, etc.). This PRD proposes a notification event tracking table that logs every notification sent, along with the user/device context at the time of sending and its full delivery lifecycle (sent → delivered → seen → clicked → landed on correct page → failed).

This is an analytics + debugging table, not the notification queue/dispatch table itself. It sits downstream of the actual send pipeline and is written to at each lifecycle stage.

2. Problem Statement

Today, if a notification underperforms, we can't tell whether it's because:

- A specific user segment doesn't engage with that notification type (e.g., premium vs free)
- A specific device/OS/app version is silently failing to deliver
- The notification text itself is misleading or weak
- The landing page is broken for that notification type
- Permissions are off for a chunk of the user base and we're wasting sends

Without a structured log tying who + what + when + outcome together, every investigation today is guesswork or requires pulling logs from multiple disconnected systems.

3. Goals

- Create a single source of truth table capturing every notification event and its full lifecycle.
- Enable segment-level analysis (gender, user type, activity status, intent level, profile quality, device/OS).
- Enable root-cause debugging of failures (permission off, token issue, missing object, duplicate sends).
- Enable A/B testing / copy testing on notification text.

- Support dashboards for delivery rate, seen rate, click rate, and mis-routing rate by notification type/category.

4. Non-Goals

- This table does not replace the notification dispatch/queue system (i.e., it doesn't decide when to send).
- This is not a real-time alerting system — it feeds reporting/BI and debugging, not live paging.
- This does not define the send-time scheduling logic (DND windows, batching, timezone handling) — that's a separate system this table can help evaluate after the fact.

5. Key Users of This Data

User	Use case
Growth / Product	Which notification types drive engagement per segment
Engineering	Diagnose delivery failures by app version / device brand
CRM / Retention team	Reactivation notification effectiveness on dormant users
Copywriting / Marketing	A/B test notification text
Support	Investigate “I didn’t get a notification” complaints

6. Functional Requirements — Data Dictionary

6.1 User & Profile Context (snapshot at send time)

Field	Example	Purpose
Member ID	2454011	Identify recipient
Gender	Male / Female	Compare response by gender
Profile created by	Self, Mother, Father, Brother, Sister, Relative/Friend	Parent-created profiles may need different notification behavior
Profile approval status	Approved / Pending / Rejected	Unapproved users may be excluded from some notifications
User type	Free / Premium / Trial / Expired Premium	Free vs premium shouldn’t get identical notifications
User activity status	Active today / Active last 3 days / Dormant	Distinguish notifying active vs inactive users
User intent level	High / Medium / Low	Users who opened plan/chat/contact are higher intent

Field	Example	Purpose
Profile quality	Good / Average / Poor	Poor-profile users may need profile-completion nudges instead of match alerts

Design note: *these attributes must be captured as a snapshot at send time, not joined live from the user's current profile. A user's activity status or user type can change after the notification was sent, and doing live joins later would corrupt historical analysis (e.g., "did dormant users respond?" needs their status at send time, not today).*

6.2 Device & App Context (snapshot at send time)

Field	Example	Purpose
App version	10.6.1	Detect version-specific notification bugs
Phone type	Android / iOS	Android/iOS push behavior differs
Device brand	Xiaomi / Vivo / Samsung / iPhone	Some Android OEMs throttle/kill push delivery
Notification permission status	On / Off / Unknown	Determine if the user could have received the push at all

6.3 Notification Content

Field	Example	Purpose
Notification type	Interest received / Message received / Profile visitor / Payment failed	Compare performance across specific triggers
Notification category	Interest / Chat / Profile / Payment / Reactivation	Group similar notification types for rollup reporting
Notification text used	"Someone has shown interest..."	Identify weak/misleading copy, support copy A/B testing

6.4 Lifecycle / Event Tracking

Field	Example	Purpose
Sent time	Date+time	When it was sent
Delivered time	Date+time	Whether/when it reached the device
Seen on phone	Yes / No	Whether it actually appeared to the user
Clicked or not	Yes / No	Whether the user interacted
Click time	Date+time	Delay between delivery and click
Did it open correct page?	Yes / No	Catch landing-page/deep-link bugs

Field	Example	Purpose
Failure reason	Permission off / Token issue / Object missing / Duplicate	Root-cause failed sends

7. Proposed Schema

```
CREATE TABLE notification_events (
    notification_event_id   BIGINT PRIMARY KEY AUTO_INCREMENT,

    -- User context (snapshot at send time)
    member_id               BIGINT NOT NULL,
    gender                  ENUM('male','female'),
    profile_created_by      ENUM('self','mother','father',
                                 'brother','sister','relative_friend'),
    profile_approval_status ENUM('approved','pending','rejected'),
    user_type               ENUM('free','premium','trial','expired_premium'),
    user_activity_status    ENUM('active_today','active_last_3_days','dormant'),
    user_intent_level       ENUM('high','medium','low'),
    profile_quality         ENUM('good','average','poor'),

    -- Device/app context (snapshot at send time)
    app_version             VARCHAR(20),
    phone_type              ENUM('android','ios'),
    device_brand            VARCHAR(50),
    notification_permission_status ENUM('on','off','unknown'),

    -- Notification content
    notification_type       VARCHAR(50) NOT NULL,
    notification_category   ENUM('interest','chat','profile',
                                 'payment','reactivation'),
    notification_text       TEXT,

    -- Lifecycle
    sent_time               DATETIME NOT NULL,
    delivered_time          DATETIME NULL,
    seen_on_phone           BOOLEAN NULL,
    clicked                 BOOLEAN NULL,
    click_time              DATETIME NULL,
    opened_correct_page     BOOLEAN NULL,
    failure_reason          VARCHAR(100) NULL,

    created_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_member_id (member_id),
    INDEX idx_sent_time (sent_time),
    INDEX idx_notif_type (notification_type),
    INDEX idx_notif_category (notification_category),
    INDEX idx_failure_reason (failure_reason)
);
```

Suggested partitioning: monthly partition on sent_time given expected high write volume (every notification generates a row).

8. Data Sources / Integration Points

Field group	Source system
Member ID, gender, profile fields, user type, activity status, intent level, profile quality	User/profile service — snapshot pulled at send time
App version, phone type, device brand, permission status	Mobile app SDK / device telemetry sent with each session or push registration
Notification type, category, text	Notification service / templating engine at dispatch
Sent time	Notification dispatch service
Delivered time, seen on phone	Push provider (FCM/APNs) delivery + client SDK acknowledgment callback
Clicked, click time, opened correct page	Client-side event tracking (app analytics SDK) on notification tap
Failure reason	Push provider error callback + dispatch service error logs

9. Non-Functional Requirements

- Volume: Estimate expected daily notification volume to size storage/partitioning (needs input from notification team).
- Write pattern: High-frequency inserts at sent_time; subsequent updates to the same row for delivered/seen/clicked/failure fields (or alternatively, an event-log design where each lifecycle stage is a separate row — see Open Questions).
- Retention: Define retention period (e.g., 12–18 months raw, aggregated/rolled-up beyond that) to control storage growth.
- Privacy/PII: Member ID links to a real user; access to this table should be restricted to authorized analytics/engineering roles. Notification text should avoid storing any dynamically-inserted PII (e.g., other member's name) beyond what's needed for debugging.
- Consistency: Snapshot fields (gender, user type, etc.) must be captured at send time, not derived via live joins.

10. Success Metrics Enabled by This Table

- Delivery rate (delivered / sent) by app version, device brand, OS
- Seen rate and click-through rate by notification type/category
- Engagement difference by gender, user type, activity status, intent level
- % of sends going to users with permission off (wasted sends)
- Failure rate breakdown by reason
- Landing page mis-route rate by notification type
- Time-to-click distribution (click_time − delivered_time)

11. Open Questions

- Row model: Should each notification be a single row updated across its lifecycle (sent → delivered → seen → clicked), or should each lifecycle stage be its own event row (append-only)? Append-only is more robust for high-write systems and avoids update contention, but requires a join/aggregation layer for reporting.
- What is the expected daily/monthly notification volume, to plan partitioning and storage?
- Who owns writing to this table — is it the notification dispatch service, or a separate event-ingestion pipeline?
- Should "Notification text used" store the final rendered text (with dynamic values) or the template ID + variables (better for text A/B testing without duplicating near-identical strings)?
- Data retention and archival policy — how long does raw event-level data need to stay queryable?
- Access control — which teams/roles get read access to this table given it contains member-linked behavioral data?

12. Rollout Plan

Phase 1 — Schema & instrumentation

- Create table (schema above), get sign-off on row model (updated-row vs append-only).
- Instrument notification dispatch service to write send-time snapshot fields.
- Instrument push provider delivery callbacks.

Phase 2 — Client-side event tracking

- Instrument app SDK to log "seen," "clicked," "click time," and "opened correct page" back to the backend.

Phase 3 — Reporting

- Build dashboards for delivery rate, engagement by segment, and failure analysis.
- Backfill historical data where possible (device/app version may not be reliably backfillable).

Appendix: full source field list and rationale is based on the attached notification data requirements table.